Syntax Directed Translation–Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

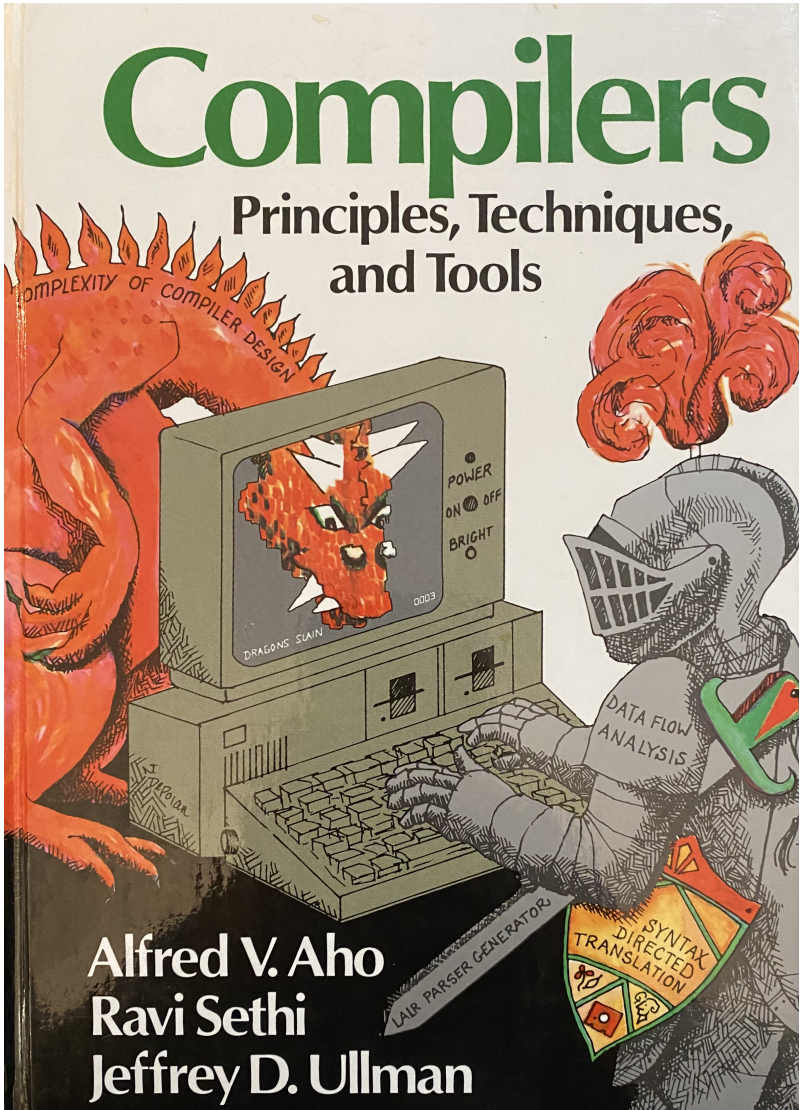
Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

Syntax Directed Translation–Steps to Work on Assignment 4



Syntax Directed Translation

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

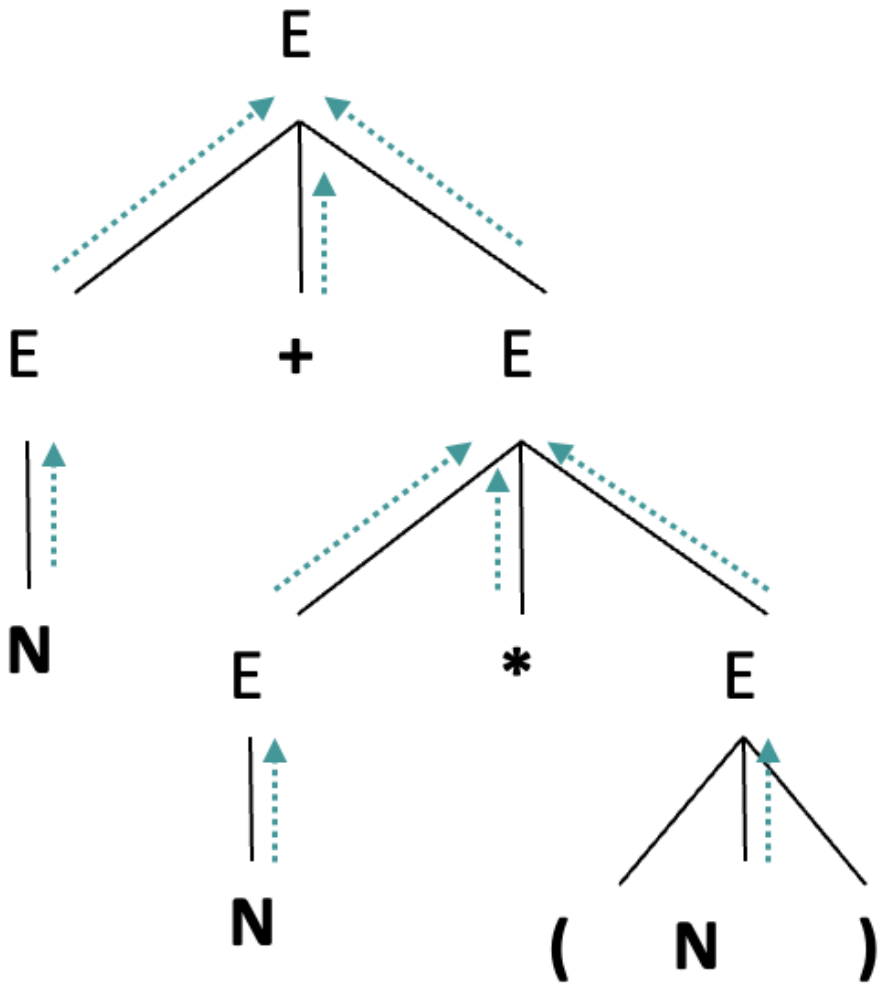
Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:



Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Change file name from A3 to A4

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

- Change A3.lex to A4.lex, A3.cup to A4.cup
- Make corresponding changes in the script file and lex file
  - Script to run:

```
>del *.class A4.java A4.output
>java JLex.Main A4.lex
>java java_cup.Main -parser A4Parser -symbols A4Symbol  A4.
    cup
>javac A4.lex.java A4Parser.java A4Symbol.java A4User.java
>java A4User
>javac A4.java
>java A4
>more A4.output
```

  - Unix commands are slightly different.

```
>rm *.class A4.java A4.output
>java java_cup.Main -parser A4Parser -symbols A4Symbol  <
    A4.cup
```

  - Note that removing A4.java and A4.output will help you debug your code
  - Those two files may be generated by earlier versions, not your current code.

Change A4.lex

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
%class A4Scanner
 ...
... return new Symbol(A4Symbol.ID) ...
 ...
```

- Do not forget to change in A4.lex
  - Scanner class is A4Scanner
  - Symbol table is A4Symbol
  - ...
- Make sure that you can run A4

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Input example

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
REAL f2(REAL x, REAL y )
BEGIN
    REAL z;
    z := x*x - y*y;
    RETURN z;
END
INT MAIN f1()
BEGIN
    INT x;
    READ(x, "A41.input");
    INT y;
    READ(y, "A42.input");
    REAL z;
    z := f2(x,y) + f2(y,x)*0.5;
    WRITE (z, "A4.output");
END
```

- Suppose that all input files contain nothing but an integer number.
- Type checking is an important component of compilers
- We skip this and many other topics ...

One possible translation

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
import java.io.*;
public class A4 {
    static BufferedReader $br;
    static BufferedWriter $bw;
    static double $tmp_real;
static double f2(double x, double y) throws Exception {
  double z;
  z=x*x-y*y;
  return z;
 }

public static void main(String [] args) throws Exception{
  int x;
  $br=new BufferedReader(new FileReader("A41.input"));
  x=new Integer($br.readLine());
  int y;
  $br=new BufferedReader(new FileReader("A42.input"));
  y=new Integer($br.readLine());;
  double z;
  z=f2(x, y)+f2(y, x)*0.5;
  $tmp_real=z;
  $bw=new BufferedWriter(new FileWriter("A4.output"));
  $bw.write(""+ $tmp_real);
  $bw.close();;
 }}
```

How to add class header

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

Wrong code

```
Program::=Program Method {: RESULT=" public class A4 {...}"; :}
 |Method {: RESULT="..."; }
```

- Where to add class header?

How to add class header

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

Wrong code

```
Program::=Program Method {: RESULT=" public class A4 {...}"; :}
 |Method {: RESULT="..."; }
```

- Where to add class header?
- You can not add it at the original top level rule as above
- That will add the header multiple times, every time the rule is applied.
- Need to add the header only once.

How to add class header

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Wrong code

```
Program::=Program Method {: RESULT=" public class A4 {...}"; :}
 |Method {: RESULT="..."; }
```

- Where to add class header?
- You can not add it at the original top level rule as above
- That will add the header multiple times, every time the rule is applied.
- Need to add the header only once.
- Solution: Add an extra rule

```
Program2::=program
```

Add a new rule

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

```
...
non terminal String Program2;
Program2::=Program:e {: RESULT=
  "public class A4 {"
  + e
  +"}"
  ;
```

- Make sure that the RESULT type is consistent with the type of the non-terminal. In this case, both the type of RESULT and Program2 are String.
- For now, do not work on the translation of other rules

Add a new rule

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
...
non terminal String Program2;
Program2::=Program:e {: RESULT=
  "public class A4 {"
  + e
  +"}"
  ;
```

- Make sure that the RESULT type is consistent with the type of the non-terminal. In this case, both the type of RESULT and Program2 are String.
- For now, do not work on the translation of other rules

```
 Program::=Program Method {: RESULT=""; :}
  |Method {: RESULT=""; }
  :
...  ...
```

- e is the translation for **Program**
- Suppose that it is empty string for now.
- Again make sure the type of **Program** is consistent with the type of the corresponding RESULT type.
- Make sure you can run the script and a translation is generated.

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Input and output

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

```
REAL f2(REAL x, REAL y )
BEGIN
    REAL z;
    z := x*x - y*y;
    RETURN z;
END
```

```
static double f2(double x, double
      y) throws Exception
{    double z;
  z=x*x-y*y;
  return z;
 }
```

- Differences
  - translate **real** to **double**, **BEGIN** to {, etc.
  - add **static**
  - Maybe you need to throw Exceptions when there are read/write operations
- Suppose your rule is:

```
method::= TYPE ID LPAREN FormalParameters RPAREN
  Block
```

- Your new rule can be as follows:

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4 {:RESULT=
  e1+ " " + e2 + "(" +e3 +")" +e4;
  :}
  ;
```

- Make sure all the symbols with labels have appropriate values returned.

A common error

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4 {:RESULT=
  e1+ " " + e2 + "(" +e3 +")" +e4;
  :}
  ;
```

- String concatenations can be wrong, especially quotes are not paired properly;

A common error

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4 {:RESULT=
  e1+ " " + e2 + "(" +e3 +")" +e4;
  :}
  ;
```

- String concatenations can be wrong, especially quotes are not paired properly;
- Align them nicely as below:

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4
  {:RESULT=
  e1
  + " "
  + e2
  + "("
  + e3
  + ")"
  + e4;
  :}
  ;
```

A common error

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4 {:RESULT=
  e1+ " " + e2 + "(" +e3 +")" +e4;
  :}
  ;
```

- String concatenations can be wrong, especially quotes are not paired properly;
- Align them nicely as below:

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4
  {:RESULT=
  e1
  + " "
  + e2
  + "("
  + e3
  + ")"
  + e4;
  :}
  ;
```

- How e2 (ID, a terminal) is translated?

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Translate a terminal

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Old A4.lex

```
<YYINITIAL >{ID} {return new Symbol(A4Symbol.ID); }
```

- Previously we return the token type only
- Now we need to have the exact ID

New A4.lex

```
<YYINITIAL >{ID} {return new Symbol(A4Symbol.ID, yytext()); }
```

Translate a terminal

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

Old A4.lex

```
<YYINITIAL >{ID} {return new Symbol(A4Symbol.ID); }
```

- Previously we return the token type only
- Now we need to have the exact ID

New A4.lex

```
<YYINITIAL >{ID} {return new Symbol(A4Symbol.ID, yytext()); }
```

- The **Symbol** constructor can take a value for the Symbol.
- In this case, the value is yytext()
- yytext() returns the string that matches with the ID regular expression.
- this is also the value for e2 in the cup file

```
method::= TYPE:e1 ID:e2 LPAREN FormalParameters:e3 RPAREN
  Block:e4
```

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Translate TYPE

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

- Suppose that your grammar is as below for TYPE:

Old A4.cup

```
TYPE::=STRING
    |INT
    |REAL
    ;
```

- The the action code can be like below:

Old A4.cup

```
TYPE::=STRING  {:RESULT="String";  :}
    |INT  {:RESULT="int";  :}
    |REAL  {:RESULT="double";  :}
    ;
```

- Note that you need to declare the type for those terminals and non-terminals as String

Translate TYPE: what if your TYPE is a terminal

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

- TYPE could be a terminal in your A3 for shorter code:

Old A4.lex

```
<YYINITIAL>STRING|INT|REAL {return new Symbol{new A4Symbol.TYPE);}
```

- You may want to change to the following to facilitate the translation:

New A4.lex

```
<YYINITIAL>STRING {return new Symbol{new A4Symbol.STRING);}
<YYINITIAL>INT {return new Symbol{new A4Symbol.INT);}
<YYINITIAL>REAL {return new Symbol{new A4Symbol.REAL);}
```

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Translate a statement

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

- How to translate assignment statement?
- You code in A3:

```
assignStmt::= IDENTIFIER EQ expression SEMI {:   :}
```

Translate a statement

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

- How to translate assignment statement?
- You code in A3:

```
assignStmt::= IDENTIFIER EQ expression SEMI {:   :}
```

- Your code in A4:

```
assignStmt::= IDENTIFIER:e1 EQ expression:e2 SEMI
    {: RESULT=
    e1
    +"="
    +e2
    +";\n";
    :}
```

- Remember to declare the right types for non-terminals (here it is a String).

Translate expressions

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

- Translation in Assignment Statement depends on the translation of Expression.
- How to translate expression?
- Suppose that the grammar is like below:
  - ```
    E::=E PLUS E {: :}
       | NUMBER
    ```
  - If not, you may want to simplify your grammar to facilitate the translation
- the translation is done recursively:

```
E::=E:e1 PLUS E:e2 {: RESULT=
  e1
  + "+"
  + e2;
  :}
  | NUMBER:e {: RESULT=e; :}
```

- in A4.lex file, return the actual text of the number

```
[0-9]+ { return new Symbol(new A4Symbol.NUMBER, yytext()); }
```
```

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Main method

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
method ::=TYPE MAIN ID LP FP RP BLOCK
```

```
INT MAIN f1 { ... } BEGIN ...END
```

- It is different from a Java main method
- Make it a correct java method
  - ignore **INT**, **ID**,
  - add **public, static, void**, arguments for main.

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

valid Java code

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

TINY program

```
READ(x, "A41.input");
```

Corresponding Java code

```
import java.io.*;
public class A4 {
...
  static BufferedReader br;
  br=new BufferedReader(new FileReader("A41.input"));
   x=new Integer(br.readLine());
```

- need to add several lines:
  - import

    ```
    import java.io.*;
    ```

  - This should be added once
  - Add it with the newly introduced rule

    ```
    Program2::=Program:e {:
      RESULT=
      "import java.io.*"
      + ....
    ```

valid Java code

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
import java.io.*;
public class A4 {
...
  static BufferedReader br;
  br=new BufferedReader(new FileReader("A41.input"));
   x=new Integer(br.readLine());
```

- add variable initiaization

```
        static BufferedReader br;
```

- Connect with the file name

```
    br=new BufferedReader(new FileReader("A41.input"));
```

- read the integer (assume always an integer)

```
      br=new Integer(br.readLine());
```

valid Java code

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

```
import java.io.*;
public class A4 {
...
  static BufferedReader br;
  br=new BufferedReader(new FileReader("A41.input"));
   x=new Integer(br.readLine());
```

- add variable initiaization

```
        static BufferedReader br;
```

- Connect with the file name

```
    br=new BufferedReader(new FileReader("A41.input"));
```

- read the integer (assume always an integer)

```
      br=new Integer(br.readLine());
```

- what if the input has multiple READ stmt:

```
    READ(x, "A41.input");
    READ(y, "A42.input");
```

Multiple READ/WRITE

Incorrect translation

```
import java.io.*;
public class A4 {
...
  static BufferedReader br;
  br=new BufferedReader(new FileReader("A41.input"));
   x=new Integer(br.readLine());

  static BufferedReader br;
  br=new BufferedReader(new FileReader("A42.input"));
   y=new Integer(br.readLine());
```

Solution:

- Declare br only once
- Declare it at the very beginning

What if the input contain variable br

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

TINY program

```
READ(br, "A41.input");
```

Corresponding Java code

```
import java.io.*;
public class A4 {
   static BufferedReader br;
   ...
  br=new BufferedReader(new FileReader("A41.input"));
  br=new Integer(br.readLine());
```

What if the input contain variable br

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

TINY program

```
READ(br, "A41.input");
```

Corresponding Java code

```
import java.io.*;
public class A4 {
   static BufferedReader br;
   ...
  br=new BufferedReader(new FileReader("A41.input"));
  br=new Integer(br.readLine());
```

Solution

- change **br** to a name that is not allowed in TINY language
- so that there is never a name conflict

Outline

1 Step1: Change from A3 to A4

2 Step 2: Translation by example

3 Step 3: Translate a method

4 Step 4: Translate a terminal

5 Step 5: Translate other components

6 Step 6: Translate a statement

7 Step 7: main method

8 Step 8: Translate READ/WRITE

9 Step 9: Make it shorter

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Combine operators

Syntax Directed Translation– Steps to Work on Assignment 4

Step1: Change from A3 to A4

Step 2: Translation by example

Step 3: Translate a method

Step 4: Translate a terminal

Step 5: Translate other components

Step 6: Translate a statement

Step 7: main method

Step 8:

A translation for Expression

lex file:

```
"+"  {   return new Symbol(A4Symbol.PLUS);  }
"-"  {   return new Symbol(A4Symbol.MINUS);  }
```

cup file:

```
E::=E:e1+E:e3  {:  RESULT=e1+"+"+e3;  :}
    |E:e1-E:e3  {:  RESULT=e1+"-"+e3;  :}
```

A shorter translation

lex file:

```
"+"|"-"  {   return new Symbol(A4Symbol.OP, yytext());  }
```

cup file:

```
E::=E:e1 OP:e2 E:e3  {:  RESULT=e1+e2+e3;  :}
```